

Code Architecture & Logic

Constrained-Path Scheduler

Overview

This document provides a high-level explanation of the codebase. The system is divided into three functional layers.

Color Legend:

Static Logic & Math

The A* Search Engine

User Interface (Streamlit)

1 Part 1: The Translation Layer (Logic)

These classes handle the raw mathematics and data conversion. They do not make decisions; they only calculate outcomes.

Class: FSRModel (The Physics Engine)

What it does: Implements the *Free Spaced Repetition Scheduler* math.

Key Logic:

- Calculates how memory stability (S) changes after a study session.
- **Spacing Effect:** Harder topics yield higher stability gains when successfully recalled ($rating \geq 3$).
- **Forgetting:** If a topic is rated poor ($rating < 3$), stability is halved ($S \times 0.5$).

Class: Translator (The Human Bridge)

What it does: Converts qualitative student inputs into quantitative float values.

Key Logic:

- Maps "Nightmare" $\rightarrow$ Difficulty Score 10.0.
- Maps "Vague Recall" $\rightarrow$ Stability Score 2.0.

Why? This abstraction allows the UI to remain jargon-free while the algorithm uses precise math.

2 Part 2: The A* Scheduler (The Engine)

This is the core of the project. It uses Artificial Intelligence to explore possible schedules and find the optimal one.

Class: SchedulerState (The Timeline)

What it does: Represents a single "snapshot" in time during the search process.

Key Logic:

- Stores the current list of topics and their knowledge levels.
- `__lt__`: Defines how the Priority Queue compares two states (Depth-First bias).
- `get_id()`: Creates a unique "fingerprint" of the state to check for duplicates (Visited Set).

Function: heuristic() (The Crystal Ball)

What it does: Calculates $h(n)$ for the A* Algorithm.

Key Logic:

- Estimates the "cost" remaining to reach the goal.
- **Formula:** Calculates the gap between *Current Stability* and *Target Mastery* (14.0 days).
Larger gaps = Higher Cost.

Function: optimize_schedule() (The Solver)

What it does: Executes the A* Search loop.

Key Logic:

1. **Priority Queue:** Always explores the most promising schedule first ($f = g + h$).
2. **Branching:** For every time slot, it simulates two parallel universes:
 - *Universe A*: The student studies the topic (Memory increases).
 - *Universe B*: The student skips the slot (Memory stays same).
3. **Pruning:** Uses a `visited` set to stop processing identical states twice.

3 Part 3: The Frontend (Streamlit)

The interface is built to be "No-Brainer," hiding the complexity of the A* engine.

Function: `main()` (The Dashboard)

What it does: Renders the website and handles data flow.

Key Components:

- `st.data_editor`: Displays interactive Excel-like grids for input.
- `column_config`: Transforms standard text columns into **Dropdown Menus** to prevent user typos (e.g., selecting "Hard" instead of typing it).
- **Integration:**
 1. Cleans raw data from the grids.
 2. Passes data to `Translator`.
 3. Runs `optimize_schedule()`.
 4. Displays the result in a clean `st.dataframe`.